🗺️ GMapping-SLAM Mapping

1. Course Content

- Master the complete process of building environment occupancy grid maps using the Gmapping laser SLAM algorithm.
- Control the A1 Pro robot to move and explore unknown areas via keyboard, and view mapping results in real-time in RViz2.
- Master the method for saving occupancy grid maps (`.yaml` + `.pgm`) after mapping is complete.

---

2. Gmapping Algorithm Introduction

- **Basic Principle**: Gmapping is an open-source laser SLAM algorithm based on the RBPF (Rao-Blackwellized Particle Filter) particle filter framework, using a localization and mapping separation mechanism (localize first, then update map).
- **Main Improvements**: Introduces improved proposal distribution and adaptive selective resampling, significantly reducing the required number of particles.
- **Applicable Scenarios and Characteristics**:
    - **Advantages**: Low computational overhead, strong real-time performance, and high accuracy when building small to medium-sized indoor environment maps.
    - **Notes**: Recommended single-frame laser points should be less than 1440; no global loop closure optimization mechanism, building ultra-large scenes requires increasing particle count, with memory and computing power consumption increasing accordingly.

---

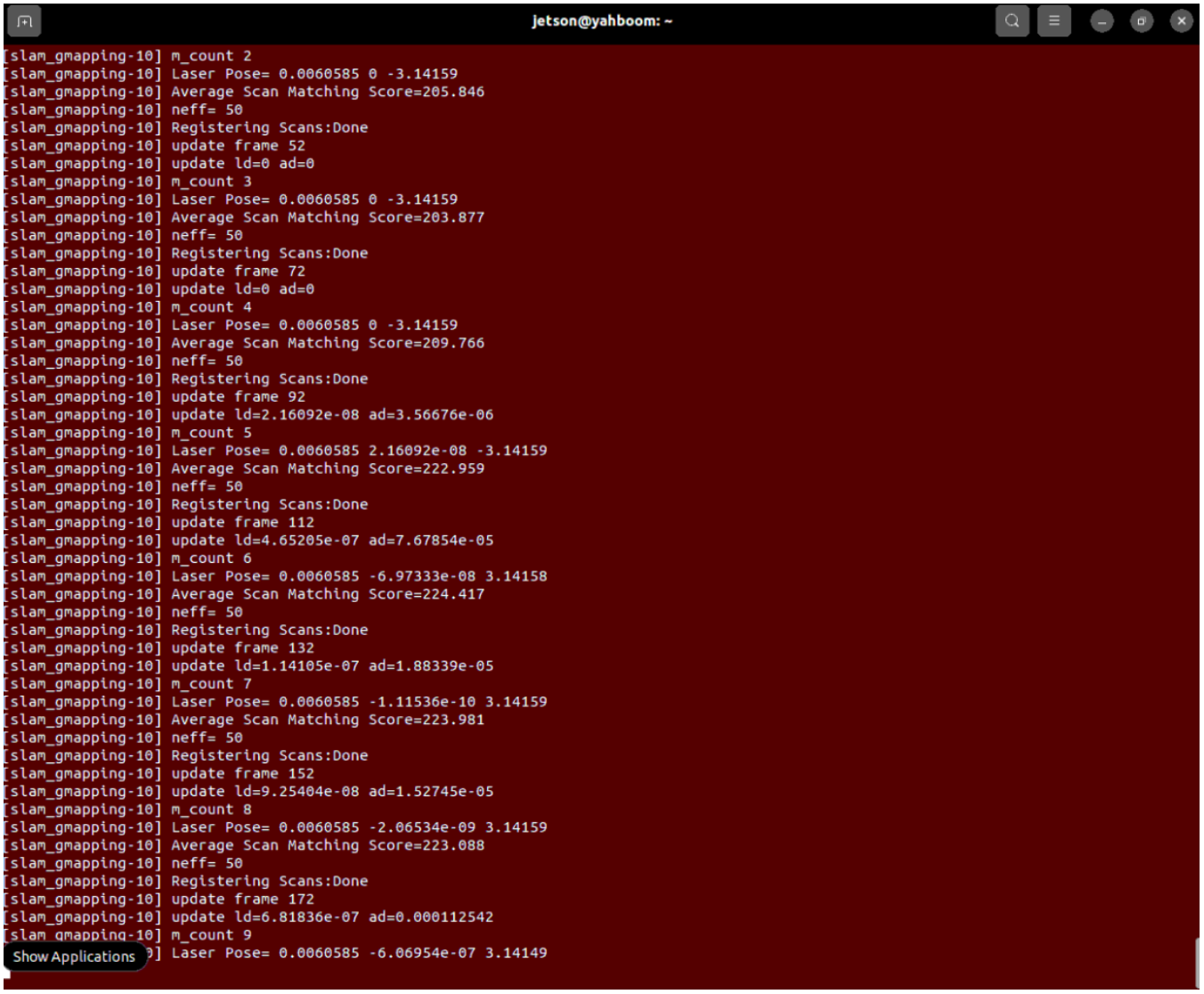
3. Preparation and Running

This example requires opening 3~4 terminals to start the following nodes in sequence:

3.1 Start Gmapping Mapping Node

Run in **Terminal 1** (automatically includes chassis motor driver, lidar driver, and Gmapping algorithm node):

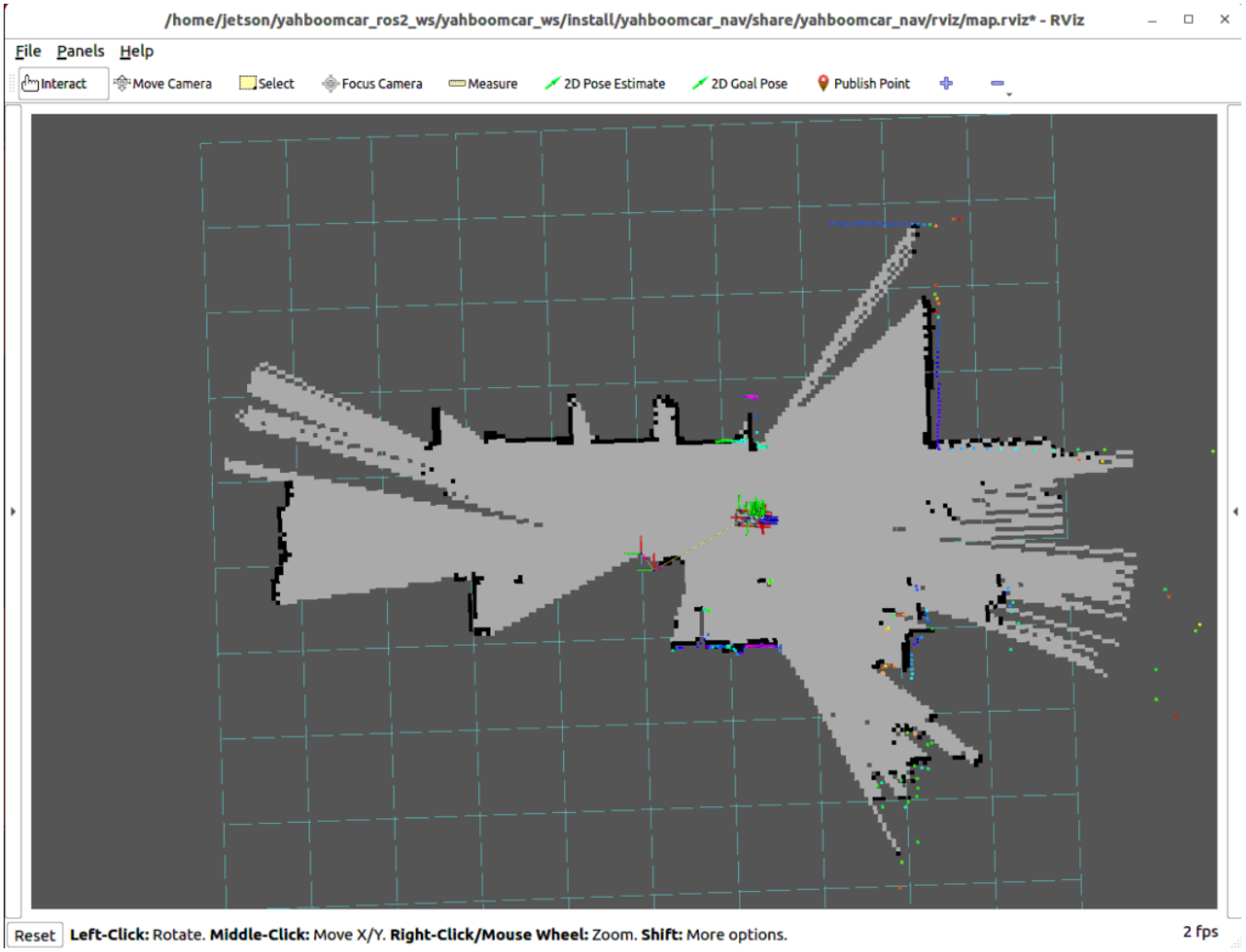
```
ros2 launch yahboomcar_nav map_gmapping_launch.py
```



3.2 Start RViz Map Visualization

Run in **Terminal 2** (it is recommended to open RViz on the virtual machine if the VM and robot are on the same LAN):

```
ros2 launch yahboomcar_nav display_map_launch.py
```



3.3 Start Keyboard Control to Move Robot for Mapping

Run in **Terminal 3** the keyboard control node:

```
ros2 run yahboomcar_ctrl yahboom_keyboard
```

```
jetson@yahboom:~/yahboomcar_ros2_ws/yahboomcar_ws$ ros2 run yahboomcar_ctrl yahboom_keyboard
1 - 6 : Select Joint 1~6
a / d : Increase (+) / Decrease (-) selected joint angle (step: 5 deg)
+ / - : Increase (+) / Decrease (-) selected joint angle (step: 5 deg)
0     : Reset all joints to initial pose

Angle Limits:
Joint 1~4 : 0° ~ 180°
Joint 5   : 0° ~ 270°
Joint 6   : 0° ~ 180°

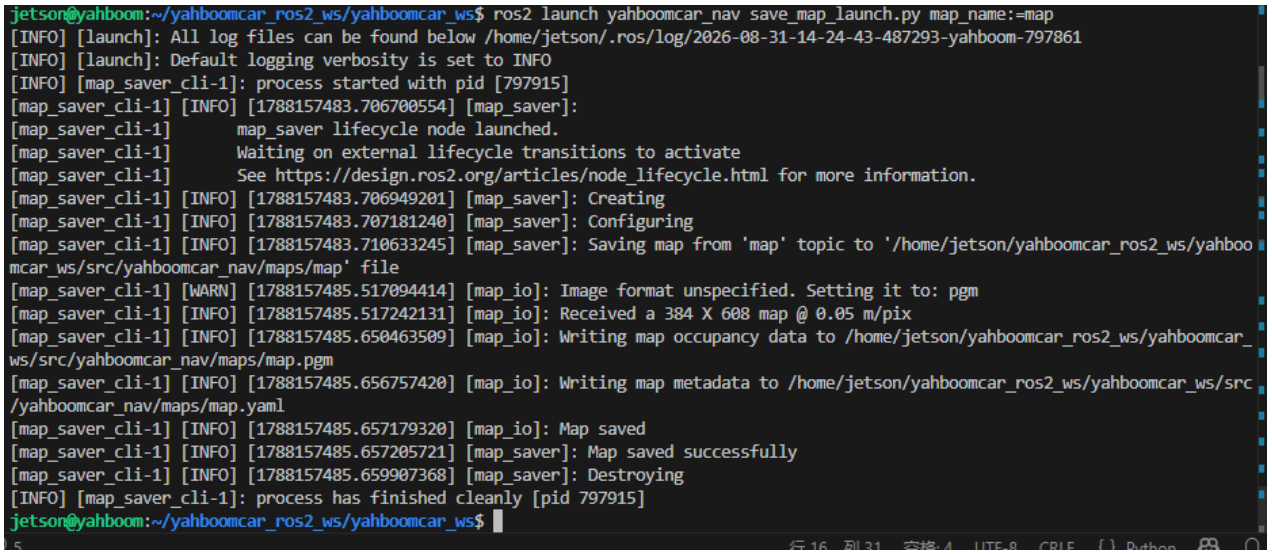
anything else : stop smoothly
CTRL-C to quit

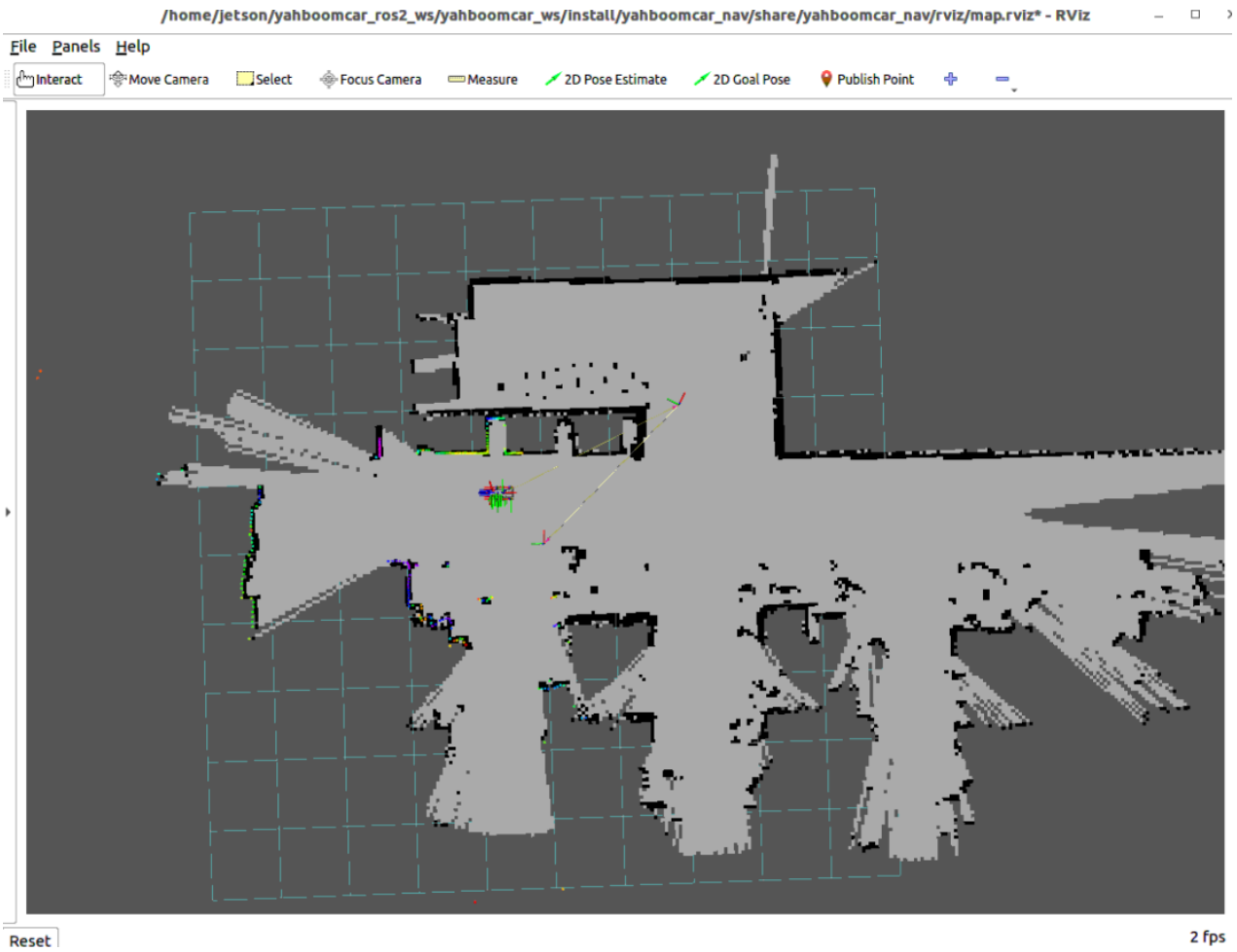
currently:      speed 0.2       turn 1.0
Current Joint Selection: Joint 1 (90°)
currently:      speed 0.18000000000000002       turn 0.9
currently:      speed 0.16200000000000003       turn 0.81
currently:      speed 0.14580000000000004       turn 0.7290000000000001
currently:      speed 0.13122000000000003       turn 0.6561000000000001
currently:      speed 0.11809800000000004       turn 0.5904900000000002
currently:      speed 0.12990780000000005       turn 0.6495390000000002
o
```

3.4 Mapping Operation and Save Instructions

1. **Remote Control Mapping Tips**:
   - Focus on the keyboard control terminal, use keyboard keys to smoothly remote control the robot (you can press `z / x` to appropriately reduce linear and angular velocity).
   - Turn slowly at corners and corridor edges, try to bring the robot back to the starting point to form a physical loop, eliminating accumulated drift.
2. **Save Occupancy Grid Map**:
   When the scanned area is complete and the outline is clear, run the map save command in **Terminal 4**:

```
ros2 launch yahboomcar_nav save_map_launch.py map_name:=map
```





Map files will be saved in the following directory by default (generating `map.yaml` and `map.pgm`):

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps/
```

> **Note**
>
> After mapping and saving are complete, press `Ctrl+C` in each terminal to stop the program.

4. Core Source Code and Algorithm Parameter Analysis

4.1 Mapping Launch File Analysis

Launch File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/launch/slam/map_gmapping_launch.py
```

```
def generate_launch_description():
    # Introduce chassis driver and lidar point cloud publishing
    laser_bringup_launch = IncludeLaunchDescription(PythonLaunchDescriptionSource(
        [os.path.join(get_package_share_directory('yahboomcar_nav'), 'launch','robot'),
         '/laser_bringup_launch.py'])
    )

    # Introduce Gmapping SLAM mapping node
    slam_gmapping_launch = IncludeLaunchDescription(PythonLaunchDescriptionSource(
```

```
        [os.path.join(get_package_share_directory('slam_gmapping'), 'launch'),
         '/slam_gmapping.launch.py'])
    )

    return LaunchDescription([laser_bringup_launch, slam_gmapping_launch])
```

- `laser_bringup_launch.py`: One-click start of A1 Pro chassis motor driver, IMU attitude solution, lidar ranging point cloud, and odometry TF transformation.
- `slam_gmapping.launch.py`: Load Gmapping algorithm core node and load configuration parameters.

4.2 Map Save Launch File Analysis

Launch File Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/launch/slam/save_map_launch.py
```

This Launch node uniformly handles map saving:

- Default save directory is set to `map_folder_dir = "~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/maps"`.
- Calls `nav2_map_server`'s `map_saver_cli` node, subscribes to `/map` topic and outputs occupancy grid matrix as standard PGM image and description file YAML:

```
    save_map = Node(
        package='nav2_map_server',
        executable='map_saver_cli',
        parameters=[{
            'map_subscribe_transient_local': False,
            'save_map_timeout': 10.0,
        }],
        arguments=[
            '-f', PathJoinSubstitution([map_save_dir, map_name]),
            '--free', '0.196',
            '--occ', '0.65'],
        condition=IfCondition(PythonExpression(["'", map_type, "' == 'gridmap'"]))
    )
```

4.3 Core Algorithm and Parameter Interpretation

Parameter Configuration File Path:

```
$HOME/yahboomcar_ros2_ws/software/library_ws/src/ros2_gmapping/slam_gmapping/params/slam_gmapping.yaml
```

Core Parameter Description Table

Parameter Name	Current Config Value	Core Function and Physical Meaning
`linearUpdate`	`1.0`	Robot triggers one map and particle update when moving straight this distance (meters)
`angularUpdate`	`0.5`	Robot triggers one map and particle update when rotating this angle (radians)
`particles`	`50`	Total number of particles in the particle filter (more particles make mapping more stable but CPU usage increases)
`delta`	`0.05`	Output grid map resolution (meters/pixel, i.e., 5cm per grid)
`maxUrange`	`5.0`	Maximum effective distance of lidar for real-time grid update (meters)
`maxRange`	`8.0`	Maximum clipping distance of lidar point cloud (meters)
`minimum_score`	`0.1`	Laser scan matching score threshold, used to determine if matching is credible
`map_update_interval`	`5.0`	Time interval for map to global publishing update (seconds)
`temporalUpdate`	`1.0`	Even if the robot is stationary, triggers periodic update every this time (seconds)
`resampleThreshold`	`0.5`	Particle resampling threshold; performs resampling when effective particle count falls below this ratio

Parameter File Core Code Snippet

```
/slam_gmapping:
  ros__parameters:
    angularUpdate: 0.5           # Rotation update threshold (radians)
    linearUpdate: 1.0            # Linear displacement update threshold (meters)
    temporalUpdate: 1.0          # Periodic time update (seconds)
    particles: 50                # Particle filter particle count
    delta: 0.05                  # Map grid resolution (meters/pixel)
    map_update_interval: 5.0     # Map update publishing period (seconds)
    maxUrange: 5.0               # Maximum effective laser distance for mapping (meters)
    maxRange: 8.0                # Laser maximum truncation distance (meters)
    minimum_score: 0.1           # Scan matching minimum confidence score
    occ_thresh: 0.25             # Grid occupancy probability threshold
    resampleThreshold: 0.5       # Particle resampling threshold
    base_frame: base_footprint   # Robot base coordinate frame
    map_frame: map               # Global map coordinate frame
    odom_frame: odom             # Odometry coordinate frame
```